



---

# TRABAJO PRÁCTICO 4: PROGRAMACIÓN ORIENTADA A OBJETOS

---



# **Resolución de los ejercicios:**

## **Creación de la clase Empleado**

Creamos la clase “Empleado” que representa a un trabajador en la empresa. Esta clase incluye:

- **Atributos de instancia:** id, nombre, puesto, salario.
- **Atributo estático:** totalEmpleados (contador global).
- **Constructores sobrecargados:**
  1. Constructor completo que recibe todos los atributos.
  2. Constructor alternativo que recibe solo nombre y puesto, asigna un id automático y un salario por defecto.
- **Métodos sobrecargados actualizarSalario():**
  - Uno que recibe un porcentaje de aumento.
  - Otro que recibe una cantidad fija.
- **Método toString():** devuelve una representación legible del objeto.
- **Método estático mostrarTotalEmpleados():** devuelve el total de empleados creados.

### **Líneas de la clase Empleado completa:**

```
public class Empleado {  
    // Atributos de la clase  
    private int id;  
    private String nombre;  
    private String puesto;  
    private double salario;  
  
    // Atributo estático (compartido por todos los empleados)  
    private static int totalEmpleados = 0;  
  
    // Constructor completo (recibe todos los atributos)  
    public Empleado(int id, String nombre, String puesto, double salario) {  
        this.id = id;           // "this" se usa para diferenciar atributos de parámetros  
        this.nombre = nombre;  
        this.puesto = puesto;  
    }  
}
```

```

        this.salario = salario;
        totalEmpleados++;
    }

    // Constructor sobrecargado (solo nombre y puesto)
    public Empleado(String nombre, String puesto) {
        this.id = totalEmpleados + 1; // Asigna id automático
        this.nombre = nombre;
        this.puesto = puesto;
        this.salario = 50000;          // Salario por defecto
        totalEmpleados++;
    }

    // Métodos para actualizar salario
    public void actualizarSalario(double porcentaje) {
        this.salario += this.salario * (porcentaje / 100);
    }

    public void actualizarSalario(int cantidadFija) {
        this.salario += cantidadFija;
    }

    // Método toString para mostrar información
    @Override
    public String toString() {
        return "Empleado{" +
            "id=" + id +
            ", nombre='" + nombre + '\'' +
            ", puesto='" + puesto + '\'' +
            ", salario=" + salario +
            '}';
    }

    // Método estático
    public static int mostrarTotalEmpleados() {
        return totalEmpleados;
    }
}

```

## Clase de prueba con método main

Creamos la clase “PruebaEmpleado” con un método main para verificar el funcionamiento de la clase Empleado.

### Acciones realizadas:

- Instanciamos empleados utilizando los dos constructores (completo y sobrecargado).
- Aplicamos aumentos de salario mediante los métodos actualizarSalario().
- Imprimimos la información de cada empleado con el método toString().
- Mostramos el total de empleados creados con el método estático mostrarTotalEmpleados().

### Fragmento de código:

```
public class PruebaEmpleado {  
    public static void main(String[] args) {  
        Empleado emp1 = new Empleado(1, "Juan Pérez", "Analista", 80000);  
        Empleado emp2 = new Empleado(2, "Ana Gómez", "Desarrolladora", 90000);  
        Empleado emp3 = new Empleado("Luis Torres", "Tester");  
        Empleado emp4 = new Empleado("María López", "Diseñadora");  
  
        emp1.actualizarSalario(10);  
        emp2.actualizarSalario(5000);  
  
        System.out.println(emp1);  
        System.out.println(emp2);  
        System.out.println(emp3);  
        System.out.println(emp4);  
  
        System.out.println("Total de empleados: " + Empleado.mostrarTotalEmpleados());  
    }  
}
```

### Salida en consola:

```
run:
Empleado{id=1, nombre='Juan Pérez', puesto='Analista', salario=88000.0}
Empleado{id=2, nombre='Ana Gómez', puesto='Desarrolladora', salario=95000.0}
Empleado{id=3, nombre='Luis Torres', puesto='Tester', salario=50000.0}
Empleado{id=4, nombre='María López', puesto='Diseñadora', salario=50000.0}
Total de empleados: 4
BUILD SUCCESSFUL (total time: 0 seconds)
```

## Conclusiones

El desarrollo del Trabajo Práctico 4 permitió aplicar los principales conceptos de la Programación Orientada a Objetos en Java:

- Comprendimos y aplicamos el uso de la palabra clave `this`, para diferenciar los parámetros de los atributos de la clase.
- Implementamos constructores sobrecargados que permiten inicializar objetos de diferentes maneras.
- Aplicamos encapsulamiento declarando los atributos como privados y gestionándolos mediante métodos públicos.
- Implementamos métodos sobrecargados `actualizarSalario()`, lo que permitió flexibilidad para actualizar el salario tanto por porcentaje como por cantidad fija.
- Definimos el método `toString()`, que facilita la representación legible de los objetos.
- Utilizamos atributos y métodos estáticos para llevar el control global del número de empleados creados.

En conclusión, el trabajo práctico permitió afianzar los fundamentos de la POO en Java, logrando un diseño modular, seguro y reutilizable. Además, la práctica de encapsulamiento contribuyó a la claridad y el mantenimiento del código.

## Link al repositorio:

<https://github.com/NazarenoAranda/UTN-TUPaD-P2-Nazareno-Aranda/tree/master/UTN-TUPaD-P2-Nazareno-Aranda>

Nazareno Aranda